# Laboratorio 1
## Reproducibilidad y Orquestación de un Producto de ML
## Motor de Reposición de Inventario

**Objetivo:**

Construir un producto de inteligencia artificial que, de forma automática y reproducible, transforme datos de ventas que llegan diariamente en una recomendación de pedido para un centro de distribución. El laboratorio integra **Git**, **DVC** y **Apache Airflow** para que cada resultado publicado pueda rastrearse hasta el código, los datos y el modelo que lo produjeron.

La complejidad principal del laboratorio no está en el modelo, sino en **operar el producto en el tiempo**: datos que llegan por partes, que se corrigen después de publicados y que deben procesarse sin intervención manual.

**Contexto:**

Una cadena de supermercados realiza sus pedidos al centro de distribución con hojas de cálculo y el criterio de cada gerente de tienda. Esto produce quiebres de stock en productos de alta rotación y merma en productos perecederos. La dirección de operaciones solicita un sistema que, cada semana, sugiera cuánto pedir por tienda y familia de producto, y que informe qué tan acertadas fueron las sugerencias anteriores.

Se utilizarán los datos públicos de **Corporación Favorita** (competencia de Kaggle *Store Sales – Time Series Forecasting*): ventas diarias por tienda y familia de producto, promociones, transacciones, precio del petróleo, feriados e información de las tiendas. Cada estudiante debe descargar los datos desde:
`https://www.kaggle.com/competitions/store-sales-time-series-forecasting/data`

Se utilizarán los archivos `train.csv`, `stores.csv`, `holidays_events.csv`, `oil.csv` y `transactions.csv`. Los archivos `test.csv` y `sample_submission.csv` corresponden a la competencia de Kaggle y no forman parte del laboratorio.

**Descripción del Producto:**

En cada fecha de corte, el producto debe entregar al menos:

1. **Pronóstico de ventas** diario para un horizonte definido por el estudiante, por tienda y familia de producto, que exprese la incertidumbre (por ejemplo, mediante cuantiles).
2. **Pedido sugerido** derivado del pronóstico y de los costos de negocio (costo de quedarse sin producto frente a costo de merma). Estos costos deben definirse en un archivo de configuración versionado, de modo que un cambio en ellos modifique la recomendación sin modificar el código.
3. **Seguimiento del desempeño**: comparación de los pronósticos ya publicados con las ventas reales a medida que estas se conocen.

**Datos y Sistema de Origen:**

A partir de los datos descargados, cada estudiante debe construir:

- Una **carga histórica** inicial.
- Una **bandeja** que simula el sistema de origen de la empresa: archivos que llegan periódicamente, uno por fecha y por fuente.

Las fuentes no llegan todas con la misma frecuencia ni con la misma anticipación, y algunos datos pueden corregirse después de haber sido entregados. El flujo debe tratar la bandeja como un sistema externo: la lee, pero nunca la modifica.

**Principio fundamental:** toda predicción debe construirse únicamente con la información que estaba disponible en el momento en que se emite.

**Requerimientos:**

**1. Organización del Proyecto y Git**

- Repositorio en GitHub con una estructura clara de código, configuración, DAGs y pruebas.
- Trabajo mediante ramas y Pull Requests; la rama principal debe estar protegida.
- Pruebas automatizadas (`pytest`) ejecutadas en cada Pull Request mediante GitHub Actions.
- Archivo de contexto para el agente de programación (`CLAUDE.md`, `AGENTS.md` o equivalente) con las reglas del proyecto.

**2. Versionado de Datos con DVC**

- Ningún dato ni modelo debe almacenarse en Git; deben versionarse con DVC y un remoto.
- Deben versionarse, como mínimo, los datos de entrada, los datos procesados, los modelos y los pronósticos publicados.
- Debe ser posible reproducir un pronóstico publicado en el pasado a partir de su versión de código y de datos.

**3. Orquestación con Apache Airflow**

- Airflow 3 ejecutado con Docker Compose.
- Flujos (DAGs) que cubran, al menos: ingesta y validación de las fuentes, preparación de los datos para el modelo, generación periódica del pronóstico y del pedido sugerido, y seguimiento del desempeño.
- Las tareas deben ser **idempotentes**: ejecutar una tarea varias veces debe producir el mismo resultado que ejecutarla una vez.
- Los datos inválidos deben detenerse antes de contaminar el resultado; las situaciones esperadas del negocio no deben provocar fallas.

- El flujo debe soportar *backfill* sobre periodos pasados y el reprocesamiento de datos corregidos.
- La lógica debe residir en módulos de Python con pruebas; los DAGs solo orquestan.

**4. Modelado**

- Un modelo de referencia (*baseline*) simple que sirva como piso de comparación.
- Al menos un modelo de machine learning que mejore al baseline. El uso de modelos de deep learning es opcional y debe justificarse en términos de costo y beneficio.
- Validación con un esquema temporal, igual para todos los modelos comparados.
- Métricas técnicas del pronóstico y una métrica de negocio basada en los costos definidos en la configuración.
- Un criterio explícito para decidir cuándo un modelo nuevo reemplaza al modelo vigente.

**5. Documentación y Decisiones**

- `README.md` con las instrucciones para levantar y ejecutar el producto desde cero.
- `DECISIONES.md` con las decisiones técnicas y de producto tomadas por el estudiante, sus alternativas y su justificación, en particular las relacionadas con la disponibilidad de la información en el tiempo, la validación de los datos y el manejo de correcciones.

## Uso de Herramientas de Desarrollo Asistido por IA:

Se permite y se recomienda el uso de Claude Code, Codex, OpenCode u otras herramientas similares. Cada estudiante es responsable de comprender, verificar y poder justificar todo el código y todas las decisiones del proyecto, independientemente de quién o qué los haya generado.

## Evaluación del Funcionamiento:

Para evaluar el producto, el equipo docente clonará el repositorio, colocará los archivos de Kaggle en la ruta indicada en el `README.md`, levantará Airflow siguiendo sus instrucciones y ejecutará el flujo sobre un periodo de datos **no publicado**. Este periodo puede incluir situaciones que se presentan en la operación real de un sistema de datos. Se evaluará que el producto funcione sin intervención manual, que reaccione correctamente ante esas situaciones y la calidad de sus recomendaciones.

## Entregables:

En el GES se entregará únicamente el **enlace al repositorio de GitHub**. El repositorio debe contener:

- El código completo, los archivos `.dvc`, la configuración y los DAGs.
- Un tag `entrega-lab1` que identifique la versión entregada.
- `README.md` y `DECISIONES.md`.

- Evidencia de funcionamiento: ejecuciones en la interfaz de Airflow, comparación de modelos y reporte de seguimiento del desempeño.

El repositorio debe ser accesible para el equipo docente. El `README.md` debe incluir las instrucciones para reconstruir el producto a partir de los archivos originales de Kaggle y la evidencia de la reproducción de un pronóstico pasado con DVC (comandos utilizados y resultados obtenidos). Se evaluará la versión correspondiente al último commit anterior a la fecha límite de entrega.